Practical 1: Linux Environment and File Management

1. AIM

Perform file and directory operations in Linux using command-line utilities, and demonstrate file permissions, ownership management, and process monitoring.

2. THEORY

Linux operates on a hierarchical tree-like directory structure originating from a single root directory denoted by /. In this architecture, virtually every system resource, hardware device, and configuration setting is abstracted and treated as a file.

- **File Permissions & Security:** Access control is managed through three distinct security classes: User (owner), Group, and Others. Permissions are defined using read (r = 4), write (w = 2), and execute (x = 1) bitmasks, which can be represented numerically in octal format (e.g., chmod 744).
- **Process Management:** In a multitasking Linux environment, every executing program is instantiated as a process assigned a unique integer identifier known as a Process ID (PID) and a Parent Process ID (PPID). System administrators utilize monitoring tools like ps (process status), top (dynamic real-time monitor), and signal dispatch commands like kill to inspect, track, and control system execution flows.

3. PROCEDURE

1. Open the Ubuntu terminal.
2. Create and navigate into a dedicated workspace: mkdir -p OS_Lab_P1 && cd OS_Lab_P1.
3. Generate sample files and inspect detailed attributes: touch file1.txt file2.txt file3.txt && ls -la.
4. Modify access permissions for a target file: chmod 744 file1.txt.
5. Spawn a background test process, locate its PID, and terminate it: sleep 500 &, ps -aux | grep sleep, followed by kill <PID>.

4. ALGORITHM

- **Step 1:** Initialize the terminal session and create a working directory context.
- **Step 2:** Execute file manipulation system utilities (touch, mkdir) and query directory contents via ls -la.
- **Step 3:** Apply octal permission masks to files utilizing the chmod command.
- **Step 4:** Initiate background execution tasks to evaluate process monitoring (ps, top) and signal termination (kill).

5. EXPECTED OUTPUT

- Directory creation succeeds, and ls -la displays correct file permissions, ownership strings, byte sizes, and timestamps.
- chmod 744 grants read, write, and execute privileges to the file owner, while restricting group and other users to read-only access.
- The background process outputs a valid PID, appears in ps search tables, and terminates immediately upon receiving the kill signal.

6. CONCLUSION

Basic Linux file manipulation commands, permission modifications (chmod), ownership structures, and process monitoring utilities (ps, kill) were successfully executed and verified on

Practical 2: Shell Scripting and System Automation

1. AIM

Develop shell scripts using decision-making statements, loops, functions, and file-handling commands to automate system tasks.

2. THEORY

Shell scripting provides a powerful mechanism for automating repetitive operating system administration, file processing, and execution routines through the Bourne Again SHell (Bash).

- **Control Flow & Logic:** Bash scripts support conditional branching structures (if-elif-else, case) to evaluate numeric and string expressions, as well as iterative looping constructs (for, while, until) to handle repetitive block execution.
- **Modularization & Text Processing:** Scripts can be structured using user-defined functions with parameter passing, local scoping, and return codes. Furthermore, integrated text filters and file-handling commands (grep, awk, sed, cat, wc) allow administrators to parse logs, inspect system states, and automate large-scale text transformations efficiently.

3. PROCEDURE

1. Open the terminal and launch a text editor to create a script file: nano practical2.sh.
2. Write the automation script incorporating control structures or looping constructs.
3. Save the file and assign executable permissions: chmod +x practical2.sh.
4. Execute the script in the terminal environment: ./practical2.sh.

4. ALGORITHM

- **Step 1:** Specify the script interpreter directive (#!/bin/bash) at the file header.
- **Step 2:** Accept administrative input parameters utilizing the built-in read command.
- **Step 3:** Implement conditional logic blocks (case or if-else) to direct execution paths.
- **Step 4:** Incorporate iterative loops (for/while) to process sequences or repetitive workflows.
- **Step 5:** Output execution logs and status messages to standard output.

5. EXPECTED OUTPUT

- Interactive command prompts successfully capture user entries and evaluate arithmetic or string conditions.
- Conditional blocks accurately classify inputs (e.g., determining even/odd numbers or executing menu choices).
- Automated file creation, verification, and text echoing execute without syntax errors.

6. CONCLUSION

Bash shell scripting was successfully leveraged to design modular, automated workflows utilizing control structures, looping constructs, and file management commands.

Practical 3: Process Creation, CPU Scheduling and Performance Analysis

1. AIM

Develop programs using system calls for process creation (fork) and implement CPU scheduling algorithms (FCFS, SJF, Round Robin).

2. THEORY

Process management forms the core of operating system design.

- **Process Creation:** The fork() system call creates a new child process by duplicating the parent's address space using copy-on-write semantics. It returns 0 to the newly spawned child process and the child's unique PID to the parent process.
- **CPU Scheduling:** CPU schedulers dictate the order in which processes utilize the processor core to maximize throughput and minimize latency:
  - *First-Come, First-Served (FCFS):* Non-preemptive queuing based strictly on arrival order; simple to implement but vulnerable to the convoy effect.
  - *Shortest Job First (SJF):* Selects the process with the shortest burst time next, minimizing average waiting times.
  - *Round Robin (RR):* Preemptive scheduling allocating a fixed time quantum cyclically, ensuring fair CPU sharing and responsive interaction.

3. PROCEDURE

1. Author the respective C source code file (process.c, fcfs.c, sjf.c, or rr.c).
2. Compile the source file using GCC: gcc program.c -o program.
3. Run the compiled executable binary: ./program.
4. Provide process counts, burst times, and time quanta as prompted during execution.

4. ALGORITHM

- **Step 1:** Accept process configurations and burst duration arrays from standard input.
- **Step 2:** For FCFS/SJF, calculate waiting times sequentially: $wt[i] = wt[i-1] + bt[i-1]$.
- **Step 3:** Compute Turnaround Time ($tat = wt + bt$) and aggregate averages.
- **Step 4:** For Round Robin, cycle through ready queues using remaining burst tracking and time slice counters until completion.

5. EXPECTED OUTPUT

- Parent and child processes output distinct execution logs and corresponding PIDs.
- Formatted tables list each process alongside its Burst Time, Waiting Time, and Turnaround Time.
- Final performance metrics display computed Average Waiting Time and Average Turnaround Time.

6. CONCLUSION

Process creation via fork() and CPU scheduling algorithms (FCFS, SJF, and Round Robin) were successfully implemented, yielding comparative performance metrics.

Practical 4: Multithreading and Resource Utilization

1. AIM

Implement multithreaded programs using POSIX threads (pthreads) and analyze CPU and memory utilization under varying thread configurations.

2. THEORY

A thread represents the fundamental unit of CPU utilization and execution within a process context.

- **Multithreading Architecture:** Unlike heavyweight processes, multiple threads belonging to the same parent process share a common address space, global variables, heap memory, and open file descriptors, while maintaining independent execution stacks, program counters, and register sets.
- **POSIX Threads (Pthreads):** The POSIX standard provides robust threading APIs (pthread_create, pthread_join, pthread_self) that enable parallel execution across multi-core processors. Multithreading significantly reduces context-switching overhead and improves execution speed for parallelizable computations, though it requires careful tracking of CPU and memory footprint scaling.

3. PROCEDURE

1. Author the multithreaded C source code (thread.c or compare.c).
2. Compile the code linking the POSIX thread library flag: gcc program.c -o program -lpthread.
3. Execute the binary in the terminal: ./program.
4. Monitor CPU core utilization and resident set memory (RSS) using monitoring tools (top, htop).

4. ALGORITHM

- **Step 1:** Define the thread worker function matching the required signature void *routine(void *arg).
- **Step 2:** Declare pthread_t handles and spawn threads using pthread_create().
- **Step 3:** Execute parallel workloads concurrently within the thread routine.
- **Step 4:** Synchronize execution and prevent zombie threads using pthread_join().

5. EXPECTED OUTPUT

- Execution logs output unique thread IDs generated via pthread_self().
- Performance comparison outputs verify reduced execution time for multithreaded routines compared to sequential execution.

6. CONCLUSION

Multithreaded applications were successfully developed and executed using POSIX threads, demonstrating enhanced computational performance and parallel core utilization.

Practical 5: Inter-Process Communication and Shared Resource Management

1. AIM

Implement Inter-Process Communication (IPC) using pipes, shared memory, and message queues, and analyze communication efficiency.

2. THEORY

Operating systems isolate processes into private virtual address spaces to protect system integrity. However, cooperative applications frequently require data exchange and synchronization, which is achieved through IPC mechanisms.

- **IPC Paradigms:** Mechanisms include unnamed/named pipes (sequential byte streams), System V message queues (structured message passing managed by the kernel), and shared memory segments.
- **Performance Advantage of Shared Memory:** Shared memory establishes a common memory region mapped directly into the virtual address spaces of communicating processes. This eliminates kernel-to-user copying overhead on every data transfer, making it the fastest IPC mechanism available, albeit requiring explicit synchronization.

3. PROCEDURE

1. Write the C source program for the selected IPC mechanism (pipe_ipc.c, shmem.c, or msgqueue.c).
2. Compile the source file using GCC: gcc ipc.c -o ipc_exec.
3. Run the compiled binary: ./ipc_exec.
4. Inspect and clean up active kernel IPC resources using ipcs and ipcrm.

4. ALGORITHM

- **Step 1:** Initialize the IPC communication channel (pipe(), shmget(), or msgget() with unique keys).
- **Step 2:** The transmitting process writes data payloads into the shared IPC construct.
- **Step 3:** The receiving process reads or attaches to the memory segment to retrieve transmitted payloads.
- **Step 4:** Unlink, detach, or close descriptors to release kernel resources.

5. EXPECTED OUTPUT

- Console outputs confirm successful payload transmission (e.g., Child received: Hello from Parent or Data written: Shared Memory Example).

6. CONCLUSION

IPC mechanisms (pipes, shared memory, and message queues) were successfully deployed, establishing shared memory as the high-throughput standard for inter-process data exchange.

Practical 6: Synchronization Mechanisms for Concurrent Systems

1. AIM

Develop programs using mutex locks and counting semaphores to solve critical section and synchronization problems (Producer-Consumer, Readers-Writers).

2. THEORY

Concurrent execution introduces race conditions when multiple threads access and modify shared variables simultaneously, leading to unpredictable program states. Proper synchronization is essential to protect critical sections:

- **Mutex Locks:** Mutual exclusion locks ensure that only one thread can enter a critical section at any given time, blocking other competing threads until unlocked.
- **Semaphores:** Synchronization variables accessed via atomic wait (P) and signal (V) operations. Counting semaphores track bounded resource pools (e.g., empty/full buffer slots in the Producer-Consumer problem), ensuring safe resource allocation and deadlock-free coordination.

3. PROCEDURE

1. Author the synchronization program source code (mutex.c or prodcons.c).
2. Compile the code with POSIX thread and semaphore support flags: gcc program.c -o program -lpthread.
3. Execute the generated binary: ./program.

4. ALGORITHM

- **Step 1:** Initialize mutex locks (pthread_mutex_init) and semaphores (sem_init with initial capacity values).
- **Step 2:** In producer threads, wait for empty buffer slots (sem_wait), acquire the mutex lock, insert items, and signal full slots (sem_post).
- **Step 3:** In consumer threads, wait for full slots, acquire the mutex, extract items, and post empty slots.
- **Step 4:** Clean up and destroy synchronization primitives upon completion.

5. EXPECTED OUTPUT

- Execution traces display orderly production and consumption logs across buffer slots without buffer overflow, underflow, or race conditions.

6. CONCLUSION

Mutex locks and semaphores successfully eliminated race conditions and solved classical concurrency challenges like the Producer-Consumer problem.

Practical 7: Deadlock Detection, Avoidance, and Recovery

1. AIM

Implement the Banker's Algorithm for deadlock avoidance and simulate deadlock detection and recovery workflows.

2. THEORY

A deadlock occurs when a set of concurrent processes are permanently blocked because each process holds a resource and waits for another resource held by another process in the cycle. Operating systems handle deadlocks via two primary strategies:

- **Deadlock Avoidance (Banker's Algorithm):** A proactive approach that evaluates resource allocation vectors (Available, Max, Allocation, and Need) to ensure the system never transitions into an unsafe state.
- **Deadlock Detection & Recovery:** A reactive approach where the system permits allocation requests, periodically executes cycle-detection algorithms (e.g., wait-for graphs), and triggers recovery actions such as process termination or resource preemption when deadlocks are detected.

3. PROCEDURE

1. Write the C source code for the Banker's Algorithm (banker.c) or detection/recovery simulation (detect.c).
2. Compile the source file using GCC: gcc banker.c -o banker.
3. Run the compiled program: ./banker.

4. ALGORITHM

- **Step 1:** Define system resource vectors and matrices for maximum demand and current allocation.
- **Step 2:** Compute the Need matrix: $Need[i][j] = Max[i][j] - Allocation[i][j]$.
- **Step 3:** Simulate safety verification loops to determine if a safe execution sequence exists that satisfies all process requirements.
- **Step 4:** For detection routines, scan allocation traces for circular wait loops and simulate victim process abortion to reclaim resources.

5. EXPECTED OUTPUT

- Program execution confirms system safety states and prints valid safe execution sequences (e.g., System is in a SAFE state. Safe sequence: P1 P3 P4 P0 P2).
- Detection outputs isolate deadlocked nodes and log successful resource reclamation following victim abortion.

6. CONCLUSION

Proactive deadlock avoidance via the Banker's Algorithm and reactive detection/recovery procedures were successfully implemented and validated.

Practical 8: Memory Management and Page Replacement Strategies

1. AIM

Simulate paging/segmentation address translations and implement FIFO, LRU, and Optimal page replacement algorithms.

2. THEORY

Virtual memory allows execution of processes that reside only partially in physical memory by swapping pages between RAM and secondary storage.

- **Address Translation:** Paging divides logical memory into fixed-size pages and physical memory into frames, mapping addresses via page tables. Segmentation organizes memory into variable-sized logical units.
- **Page Replacement Algorithms:** When physical memory is full and a page fault occurs, the OS selects a victim page to evict:
  - *FIFO (First-In, First-Out):* Evicts the oldest resident page; simple but prone to Belady's Anomaly.
  - *LRU (Least Recently Used):* Evicts the page unused for the longest time, approximating optimal performance using historical access patterns.
  - *Optimal:* Evicts the page that will not be used for the longest time in the future, providing the theoretical minimum page faults.

3. PROCEDURE

1. Author the C simulation program (fifo.c, lru.c).
2. Compile the source code using GCC: gcc page.c -o page_sim.
3. Execute the binary and enter page reference strings and frame allocations when prompted.

4. ALGORITHM

- **Step 1:** Accept input reference strings and total physical frame allocations.
- **Step 2:** Iterate through each page reference, determining whether it results in a cache hit or page fault.
- **Step 3:** On page faults with full frames, apply replacement heuristics (FIFO: queue order; LRU: timestamp tracking).
- **Step 4:** Aggregate total page faults and calculate system hit ratios.

5. EXPECTED OUTPUT

- Translated physical addresses or computed page fault counts and hit ratios are displayed for each evaluated algorithm.

6. CONCLUSION

Memory address translation and page replacement algorithms were simulated, confirming that LRU and Optimal strategies yield significantly fewer page faults than FIFO.

Practical 9: File System and Disk Scheduling Analysis

1. AIM

Implement file allocation and disk scheduling algorithms (FCFS, SSTF, SCAN, C-SCAN) and analyze storage utilization and seek times.

2. THEORY

Disk I/O performance heavily depends on mechanical read/write head movement seek times. Disk scheduling algorithms optimize request servicing order:

- *FCFS:* Services requests strictly in arrival order; simple but inefficient.
- *SSTF:* Selects the request closest to the current head position, minimizing immediate seek time but risking request starvation.
- *SCAN / C-SCAN:* Elevator algorithms that sweep systematically across disk cylinders, ensuring predictable wait times and preventing starvation.
- *File Allocation:* Strategies like contiguous, linked, and indexed allocation manage disk block mapping and storage utilization.

3. PROCEDURE

1. Author the disk scheduling C source code (fcfs_disk.c, sstf_disk.c).
2. Compile the program using GCC: gcc disk.c -o disk_sim.
3. Execute the binary and provide request cylinder sequences and initial head positions.

4. ALGORITHM

- **Step 1:** Accept input request sequences and initial head positions.
- **Step 2:** Compute absolute head movement distances for FCFS sequentially.
- **Step 3:** For SSTF, dynamically evaluate unvisited requests to find the minimum distance relative to the current head.
- **Step 4:** Aggregate total head movement tracks and calculate average seek times.

5. EXPECTED OUTPUT

- Total head movement track counts and computed average seek times are displayed for each scheduling algorithm.

6. CONCLUSION

Disk scheduling algorithms were successfully implemented and evaluated, proving that SSTF optimizes immediate seek times while directional algorithms ensure fair request servicing.

Practical 10: Containerization and Resource Monitoring for AI/ML Workloads

1. AIM

Deploy a Python machine learning application inside a Docker container, monitor resource utilization using Linux tools, and evaluate containerization benefits.

2. THEORY

Containerization represents a lightweight virtualization paradigm that packages an application together with its runtime environment, binaries, and dependencies into a single isolated portable unit called a container.

- **Containers vs. Virtual Machines:** Unlike traditional virtual machines that emulate complete hardware stacks and guest operating systems, containers share the host operating system's kernel via namespace isolation and control groups (cgroups). This architecture results in faster startup times, minimal storage footprints, and high efficiency.
- **AI/ML Deployment:** Docker ensures consistent execution environments across development and production, eliminating dependency mismatches for complex machine learning libraries like scikit-learn.

3. PROCEDURE

1. Create a Python script (app.py) executing a machine learning pipeline using scikit-learn.
2. Author a Dockerfile defining base image layers, working directories, and dependency installations.
3. Build the container image using the Docker daemon: docker build -t aiml-workload:1.0 .
4. Run the container instance: docker run --name ml-container aiml-workload:1.0.
5. Monitor live resource usage using container metrics: docker stats ml-container.

4. ALGORITHM

- **Step 1:** Define application source code and dependency manifests (requirements.txt).
- **Step 2:** Author container build instructions in the Dockerfile (FROM, WORKDIR, RUN, COPY, CMD).
- **Step 3:** Build cached container image layer snapshots via the Docker engine.
- **Step 4:** Launch isolated container instances and monitor CPU and memory utilization metrics.

5. EXPECTED OUTPUT

- Terminal logs display successful model training and evaluation accuracy scores inside the container environment.
- Live docker stats output displays container CPU percentages, resident memory consumption (RSS), and network I/O statistics.

6. CONCLUSION

An AI/ML application was successfully containerized using Docker, verifying through resource monitoring tools that container deployment delivers isolation and portability with minimal performance overhead.